Prédiction Data-Driven de la Vie Utile Résiduelle

Application au jeu de données CMAPSS de la NASA

[Votre Prénom Nom]

École d'Ingénieurs – Projet de Fin de Module

2025–2026

Plan de la présentation

1. Introduction
2. Jeu de données et prétraitement
3. Métriques d'évaluation
4. Modèle XGBoost
5. Modèle LSTM
6. Comparaison et discussion
7. Démonstration Streamlit
8. Conclusion

Contexte et motivation

Maintenance prédictive :

- Anticiper les défaillances avant qu'elles ne surviennent
- Minimiser les arrêts de production imprévus
- Optimiser la planification des interventions

Problématique : Comment prédire la **Vie Utile Résiduelle** (RUL) d'un moteur turbofan à partir de données capteurs ?

Définition – RUL

Nombre d'unités de temps restantes avant qu'un composant ne puisse plus accomplir sa fonction désirée.

Objectifs du projet

1. **Explorer** le jeu de données public CMAPSS (NASA)
2. **Prétraiter** les données temporelles de dégradation de moteurs
3. **Implémenter** et comparer deux approches data-driven :
   - **XGBoost** – Gradient boosting (machine learning classique)
   - **LSTM** – Réseaux de neurones récurrents profonds
4. **Évaluer** quantitativement les performances (RMSE, S-score)
5. **Démontrer** via une interface Streamlit interactive

CMAPSS – Commercial Modular Aero-Propulsion System Simulation

Origine : NASA Prognostics Center of Excellence

Données FD001 :

- 100 moteurs turbofan simulés
- 26 colonnes par observation
- Fichiers : train, test, RUL (vérité terrain)

Structure :

- Col 1–2 : ID moteur, cycle
- Col 3–5 : Réglages opérationnels
- Col 6–26 : 21 capteurs

Tâche de prédiction

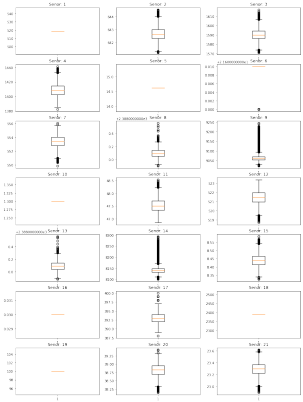
`train` → cycles jusqu'à défaillance

`test` → cycles partiels

`RUL` → vérité terrain

Objectif : Prédire la RUL pour chaque moteur test

Analyse exploratoire – Capteurs



- 21 capteurs analysés via diagrammes en boîte
- Certains capteurs présentent une variance quasi nulle

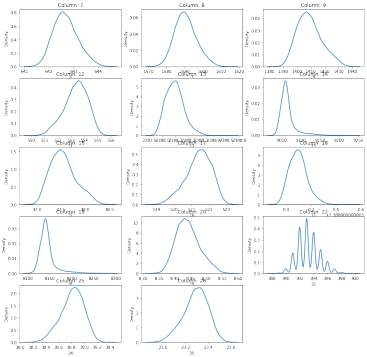
Sélection des capteurs pertinents

Capteurs écartés (constants) :

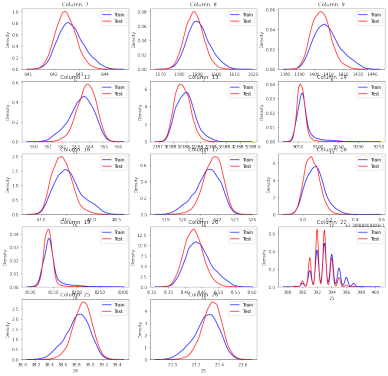
- Colonne 6, 10, 15, 21, 23, 24
- Colonne 11 (quasi-constant : 21.60 / 21.61)

Capteurs retenus : 14 capteurs

Normalisation : MinMaxScaler $[-1, 1]$ appliquée *moteur par moteur*



Similarité train / test



Constat

Les distributions des données de test (orange) sont globalement similaires à celles d'entraînement (bleu) – condition favorable à la généralisation.

Modélisation de la dégradation

Modèle linéaire :

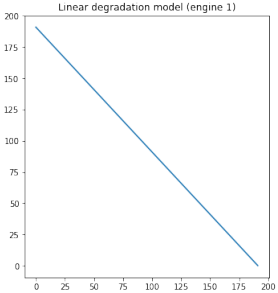
$$\mathrm{RUL}(t) = T - t$$

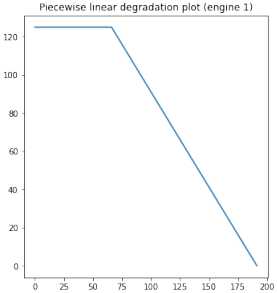
Dégradation constante d'un cycle par cycle.

Modèle linéaire par morceaux :

$$\mathrm{RUL}(t) = \begin{cases} R_{\text{early}} & \text{si } T - t > R_{\text{early}} \\ T - t & \text{sinon} \end{cases}$$

Avec $R_{\text{early}} = 125$ cycles pour FD001.





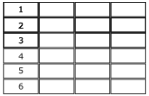
Fenêtrage temporel des séquences

Principe : Extraire des fenêtres glissantes de longueur $W = 30$ avec un chevauchement maximal ($S = 1$).

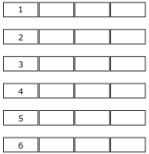
Forme des tenseurs :

- Entrée : $(B, 30, 14)$
- $B =$ nombre de fenêtres
- 30 cycles d'historique
- 14 capteurs

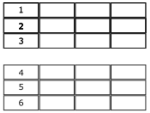
Prédiction test : Moyenne des prédictions sur les N dernières fenêtres de chaque moteur.



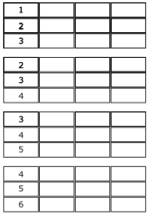
(Original Data)



(Data with window length of 1)



(Data with window length of 3 and no overlap between windows)



(Data with window length of 3 and overlap of 2 rows)

RMSE et S-score

RMSE (Root Mean Square Error)

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

Métrique principale – pénalise les grandes erreurs.

S-score (asymétrique)

$$S = \sum_{i=1}^{N} s_i$$

$$s_i = \begin{cases} e^{-d_i/13} - 1 & d_i < 0 \\ e^{d_i/10} - 1 & d_i \geq 0 \end{cases}$$

Pénalise plus les **surestimations** (risque de défaillance imprévue).

Principe de XGBoost

XGBoost = eXtreme Gradient Boosting

- Ensemble d'arbres de décision faibles
- Construction séquentielle : chaque arbre corrige les erreurs résiduelles
- Régularisation intégrée pour éviter le surapprentissage

Préparation des données :

- Aplatissement des fenêtres : $(B, 30, 14) \rightarrow (B, 420)$

Fonction objectif

$$\mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

Perte + Régularisation

Recherche d'hyperparamètres

Validation croisée 10-folds sur une grille explorant :

- `max_depth` $\in \{2, 3, 4, 5\}$
- `eta` $\in \{0.01, 0.1, 0.3, 0.5, 1.0\}$

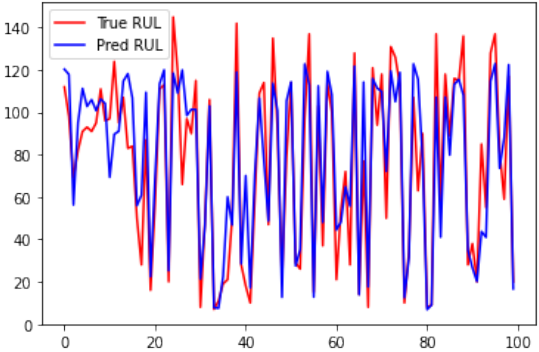
`max_depth`	`eta`	RMSE CV	Rounds
2	0.1	18.26	299
3	0.1	18.18	183
4	0.1	18.09	113
5	**0.1**	**17.96**	**95**

Meilleurs paramètres : `max_depth=5`, `eta=0.1`, 100 rounds

Résultats XGBoost sur FD001

Performances

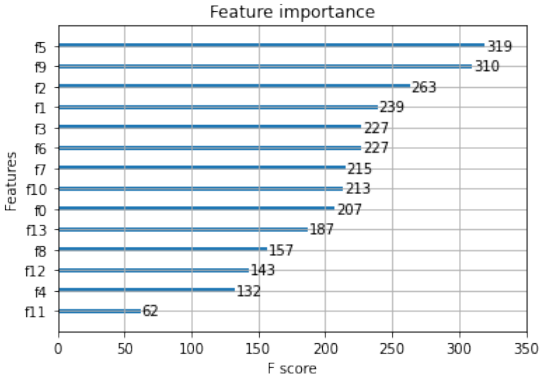
Métrique	Valeur
RMSE	**19.07** cycles
S-score	1 024.43



Constats :

- Prédictions globalement correctes
- Erreurs plus marquées sur certains moteurs atypiques
- S-score élevé $\rightarrow$ surestimations coûteuses

Importance des variables – XGBoost



Analyse :

- Certaines caractéristiques dominent la prédiction
- Les variables issues des capteurs de température et pression sont les plus informatives
- L'importance décroît rapidement après les 10 premières variables

Limite : le modèle ne capture pas explicitement la dimension temporelle.

Pourquoi les LSTM ?

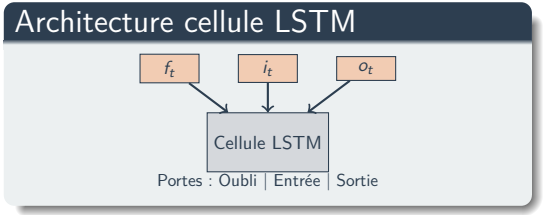
Réseaux de Neurones Récurrents (RNN)

- Conçus pour les données séquentielles
- Maintiennent une mémoire interne de l'état passé

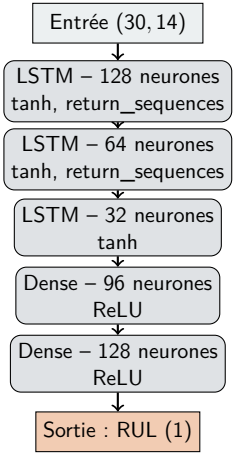
LSTM (Long Short-Term Memory)

- Résolvent le problème de disparition du gradient
- Trois portes : oubli, entrée, sortie
- Capture des dépendances **long terme**

Avantage pour CMAPSS : la dégradation est un phénomène temporel étalé sur des centaines de cycles.



Architecture du réseau LSTM implémenté



Paramètres : 223 521

Protocole d'entraînement

Optimiseur : Adam

- Taux initial : 10^{-3}
- Réduction à 10^{-4} après 5 époques

Partition :

- Train : 80%
- Validation : 20%

Configuration :

- 10 époques
- Batch size : 128
- Perte : MSE

Évolution de la perte

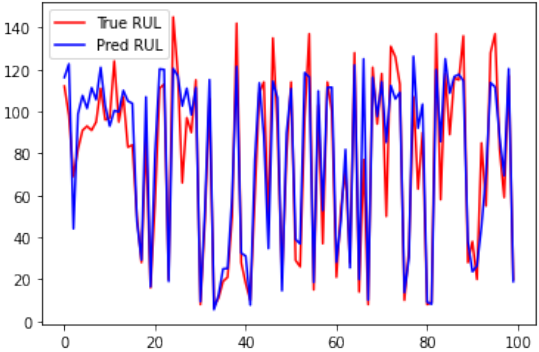
Époque	Train	Val
1	3235	959
5	210	169
10	154	**150**

Constat : L'entraînement est volontairement court (10 époques) pour éviter le surapprentissage sur le jeu de validation.

Résultats LSTM sur FD001

Performances

Métrique	Valeur
RMSE	**15.17** cycles
S-score	**448.44**



Constats :

- Très bonne adéquation à la RUL réelle
- S-score très inférieur à XGBoost
- Moins de surestimations coûteuses

Tableau comparatif XGBoost vs LSTM

Métrique	**XGBoost**	**LSTM**
RMSE (cycles)	19.07	**15.17**
S-score	1 024.43	**448.44**
Nombre de paramètres	–	223 521
Temps d'entraînement	Quelques secondes	∼100 secondes
Capacité temporelle	Non	Oui
Interprétabilité	Élevée	Faible

Synthèse

Le LSTM surpasse XGBoost de **20,4%** en RMSE et divise le S-score par plus de 2, grâce à sa capacité à modéliser les dépendances temporelles.

Discussion

Avantages LSTM

- Meilleure précision de prédiction
- Modélisation explicite du temps
- Moins de surestimations (S-score faible)

Limites

- Complexité accrue (220k+ paramètres)
- Temps d'entraînement plus long
- Non-déterminisme des opérations GPU
- Faible interprétabilité (boîte noire)

Reproductibilité : Les poids du modèle pré-entraîné sont archivés (`.h5`) pour garantir des résultats identiques.

Application de démonstration

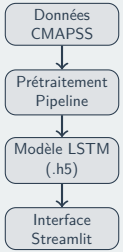
Objectif : Mettre à disposition un outil interactif de prédiction de RUL.

Fonctionnalités :

- Sélection d'un moteur parmi les 100 tests
- Visualisation des signaux capteurs
- Prédiction temps réel avec le modèle LSTM
- Affichage des métriques (RMSE, S-score)
- Comparaison True RUL vs Pred RUL interactive

Technologies : Streamlit, TensorFlow, Pandas, Plotly

Architecture



Conclusion

Réalisations :

1. Pipeline complet de prétraitement des données CMAPSS
2. Implémentation et comparaison de XGBoost et LSTM
3. Supériorité du LSTM (RMSE 15.17 vs 19.07 cycles)
4. Application Streamlit de démonstration

Perspectives :

- Extension aux sous-ensembles FD002, FD003, FD004
- Intégration de mécanismes d'attention (GRU + Attention)
- Prédiction probabiliste (distribution de RUL)
- Déploiement embarqué (quantification, pruning)

Merci de votre attention

Questions ?

Contact : [votre.email@ecole.fr]

Code source : `https://biswajitsahoo1111.github.io/rul_codes_open/`